VSB TECHNICAL UNIVERSITY OF OSTRAVA | FACULTY OF MECHANICAL ENGINEERING | DEPARTMENT OF APPLIED MECHANICS

FATIGUE TEST PROTOCOLS IN FABER COST ACTION (CA23109)

Working Group 4.7 Low-Cycle Fatigue

LCF Evaluation Toolchain

UNIAXIAL CONSTANT AMPLITUDE CASES

Authors:

Radim Halama[1], Marek Gazdačko[2], Michal Kořínek[1], Ajay Vignesh Natarajan[1], Petr Gál[3]

[1]VSB-Technical University of Ostrava, Faculty of Mechanical Engineering, Czechia
[2]Technical University of Košice, Faculty of Mechanical Engineering, Slovakia
[3]ÚJV Řež, Czechia

Last update: October 14, 2026

Table of contents

Introduction 3
LCF tool Evaluation (*LCF_single_eval*) 3
Nomenclature used in output files for both (stress/strain-control) 3
Input files for both 4
Strain controlled case 5
Files information 5
User interface 5
Outputs 9
Stress controlled 12
Files information 12
User interface 12
Outputs 16
e-N curve evaluation – conventional vs 3D method (*lcf_3d_eval*) 19
Files information 19
User interface 20
Outputs 21
Examples and Verification Cases 23
Uniaxial Ratcheting and LCF Cases (Inconel 718) 23
Multi-Specimen e-N Evaluation Case (AlSi10Mg) 23
Bibliography 24

Introduction

The fatigue post-processing workflow is structured as a modular computational pipeline comprising specialized Python tools.
First, the *LCF_single_eval* script processes raw, single-specimen time-series data exported from testing machines to extract stabilized half-life properties, determine the failure cycle, and export comprehensive single-test evaluation outputs.
Subsequently, the *input_preparation* script aggregates these individual records to compile a structured multi-test dataset.
Finally, the *lcf_3d_eval* script takes in this compiled dataset to execute multi-specimen parameter identification, comparing conventional 2D regressions against the 3D method, developed by Prof. Adam Niesłony et al. [1], under both constant and (stress/strain-level) variable elastic modulus, while fitting 4-parametric energy models (Smith-Watson-Topper and plastic work as fatigue parameters) and automatically generating a standardized, publication-ready two-page PDF test protocol.

LCF tool Evaluation (*LCF_single_eval*)

This Python tool automates the post-processing of raw experimental data recorded by universal testing machines for individual specimens. It reads continuous time-series files guided by a JSON configuration template, handling both high-frequency continuous logs and sparse data recordings (where data is captured only at selected cycles).
The script processes these records to isolate individual hysteresis loops, extracts stabilized half-life properties, and determines the fatigue life to crack initiation using robust physical threshold criteria (such as a percentage drop in stress amplitude or degradation in the elastic stiffness of the bottom hysteresis loop branch).
Finally, it exports specimen-level summary tables and detailed evolution parameters in parallel with generating a standardized, publication-ready PDF test report.

Nomenclature used in output files for both (stress/strain-control)

Legend – output values for the mid-life cycle (*output_half.csv*)

σ_a	Sig_A	stress amplitude	[MPa]
σ_m	Sig_M	mean stress	[MPa]
ε_a	Eps_A	strain amplitude	[-]
ε_m	Eps_M	mean strain	[-]
ε_{ap}	Eps_AP	plastic strain amplitude (calculated from the additive rule and Hooke's law considering averaged dynamic modulus of given cycle)	[-]
E_{upp}	E_dyn_tens	dynamic modulus in tension, upper branch) identified by a linear regression considering the interval $<N_1,N_2>$.	[MPa]
E_{btm}	E_dyn_comp	dynamic modulus in compression, bottom branch) identified by a linear regression considering the interval $<N_1,N_2>$	[MPa]

Y_{cyc}	Y_cyclic	cyclic yield stress for 0.005% plastic strain offset	[MPa]
N_f	Nf	number of cycles to failure considering 25% drop of stress amplitude	[cycles]
N_f^{E10}	Nf_E10%	number of cycles to failure considering dynamic modulus decrease by 10%	[cycles]
ΔW_p	DeltaWp	plastic work in the half-life cycle identified by a linear regression considering the interval <N1, N2>.	[MPa]
W_p^{tot}	Wp_tot	total amount of plastic work till Nf (MPa)	[MPa]
ΔW_p^{mean}	Wp_mean	averaged valued of plastic work per cycle (MPa)	[MPa]
ΔW_p^{VarE}	DeltaWp_VarE	plastic work in the half-life cycle calculated considering different dynamic elastic moduli in upper and bottom parts of hysteresis loop	[MPa]

Note: cyclic yield stress Y_{cyclic} is determined from the mid-life hysteresis loop using an offset plastic strain ε_{off} (default value is 0.05%), see Figure 1.

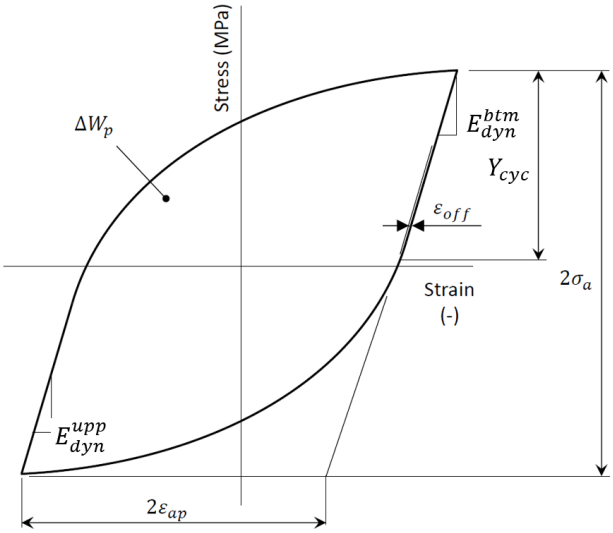


Figure 1 Saturated hysteresis loop

Input files for both

The input file with raw data used as a demonstration has the following character (example for stress-controlled loading case):

CSV file (AllData750MPa100MPa.csv):

Time (s);Total Strain (%);Plastic Strain (%);Stress (MPa);Load (kN);Position (mm);Cycle Number
0;0.00217;0.002851;0.054208;0.001538;-1.191844;1
0.0752;0.008612;0.184023;13.956172;0.395918;-1.185747;1
0.1504;0.017958;0.379528;28.76748;0.816095;-1.181965;1

Description (used columns only)

Time (s)	not necessarily starting at 0 s; lines containing data captured during gripping can be cut at the beginning of the code execution

Total Strain (%)	total engineering strain (provided either directly in percent or converted from elongation via gauge length)
Stress (MPa)	engineering axial stress (calculated directly from applied force and initial cross-sectional area, or supplied directly in the dataset)
Cycle Number (-)	identification number of the load cycle associated with the given data point

Strain controlled case

To capture the stable cyclic plastic deformation and hysteresis behavior typical of low-cycle fatigue (LCF), tests are conducted under strain control. This proposed Python tool covers raw machine data ingestion, cyclic material behavior and hysteresis loop extraction, plastic strain separation and plastic work evaluation (W_p), automated failure identification, dynamic modulus tracking, and real-time visual plot verification, concluding with standardized data export for downstream fatigue evaluation.

Files information

First of all, before starting the script, it is necessary to check if every file is located in the working directory (project folder). Here is a list of them:

- LCF_single_eval.py — main script
- Protocol_LCF.py — support script
- Config_StrainControl_In718SLMZ.json — configuration template
- AllData1d5perc.csv — experimental raw data
- Header_logo.png — support figure
- S105008959299.bib (optional) — additional reference

The structure of the input file is discussed in the previous chapter.

User interface

After running the script in the Python interpreter (Spyder is recommended; scripts have been tested on Python 3.14.6), the first prompt asks the user to select an existing *.json* configuration from the accompanying files.

```
================================================================================
     UNIFIED LCF & RATCHETING TOOL (FABER CA 23109)
================================================================================
Select existing JSON configuration profile via dialog? (Y/N) [Y]:
```

Throughout the script execution, default suggestions derived from the configuration file are displayed in square brackets, allowing the user to simply press Enter to accept them. Once the configuration is loaded, the option to select the test method becomes available.

```
Configuration mode: 'Strain-controlled'
Select test mode:
  1 = Strain-controlled
  2 = Stress-controlled (Ratcheting)
Insert your choice [1]:
```

Subsequently, the material name and its reference elastic modulus (E_0) are retriever (from *.json*), and a graphical file dialog opens to let the user select the raw experimental data file from the universal testing machine

```
Active material: 'In718' | Reference E0: 150000 MPa

--------------------------------------------------------------------------------
SELECT RAW TEST DATA FILE (.CSV or .TXT)
```

with the possibility of choosing the citation:

```
--> Found bibliography file: 'S0142112326001799.bib'
  Use 'S0142112326001799.bib' for protocol citation? (y/n) [y]:
------------------------------------------------------------------------------------------
```

To facilitate a quick verification, the first three rows are displayed to inspect and report the delimiter and decimal separator of the document

```
================================================================================
FILE: AllData1d5perc.csv
--------------------------------------------------------------------------------
HEADER: Time (s);Total Strain (%);Plastic Strain (%);Stress (MPa);Load (kN);Position (mm);Cycle Number
DATA ROW 1:  0;-0.000787;-0.000789;0.003348;9.54E-05;-0.753473;1
DATA ROW 2:  0.1502;-0.000274;-4.00E-06;-0.35576;-0.010133;-0.754809;1
================================================================================
```

followed by a final step to confirm the detected columns

```
DETECTED COLUMNS IN FILE:
  [0] Time (s)
  [1] Total Strain (%)
  [2] Plastic Strain (%)
  [3] Stress (MPa)
  [4] Load (kN)
  [5] Position (mm)
  [6] Cycle Number

CONFIRM COLUMN ASSIGNMENTS (Press [Enter] to keep default, or enter index/name):
Confirm column for Time [Time (s)]:
Confirm column for Force / Stress [Stress (MPa)]:
Confirm column for Deformation / Strain [Total Strain (%)]:
Confirm column for Cycle Number [Cycle Number]:
```

then confirming (or defining) the name of the output folder

```
Enter output folder name [Output_In718_Strain_Control]:
```

Additional information will then be requested, such as the unit format

```
Are data in the form of force/elongation (Y/N) - if N, stress and strain are expected directly [N]:
Is strain in percent (%)? (Y/N) [Y]:
```

Therefore, after plotting Figure 2, the analysis will start with the calculation of the 25% drop in stress amplitude,

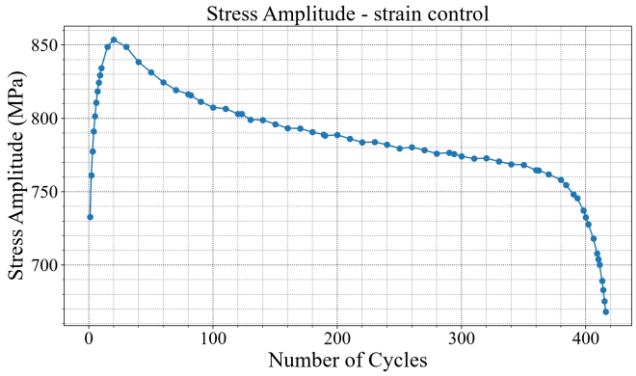


Figure 2 Stress amplitude decreasing in increasing number of cycles

selecting a range where the linear regression method will be used.

```
Define the region for linear regression (including half life)!
Enter value for N1 [160]:
Enter value for N2 [330]:
Number of cycles to failure (25% drop in Sig_A): Nf = 416
Is it OK (Y/N) [Y]:
```

First required values, N_1 and N_2, defines the interval, where the cyclic hardening/softening curve shows a linear behaviour. The interval should cover expected $N_f/2$. Then, user can confirm obtained number of cycles to failure (decrease of stress amplitude by 25%) or define different interval. In the example, $N_f = 416$ cycles.

To ensure the data is consistent, the first few rows of the data are checked,

```
================================================================================
INSPECTION OF INITIAL LOADING DATA (ZEROING / EXTENSOMETER TARE REMOVAL)
================================================================================
Total skipped ROWS so far: 3 rows from raw CSV (rows_to_skip in JSON)).
Please, interrupt the evaluation and change the value in JSON if initial data is missing!
First stress peak is at row index: 50 (703.8 MPa).
[INFO] No sharp tare jump detected. Loading appears smooth from Row 0.
Enter number of initial rows to trim. Write 0 to finish [0]:
```

and the first branch of the hysteresis loop is displayed, as shown in Figure 3.

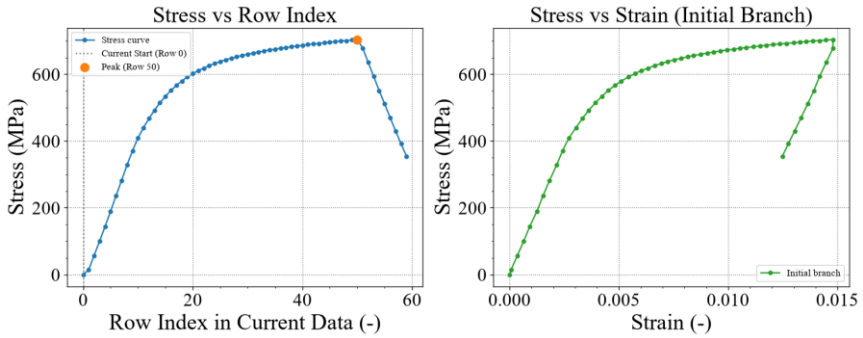


Figure 3 Initial data displayed for cutting

Once confirmed, the first 10 hysteresis loops are plotted, as shown in Figure 4.

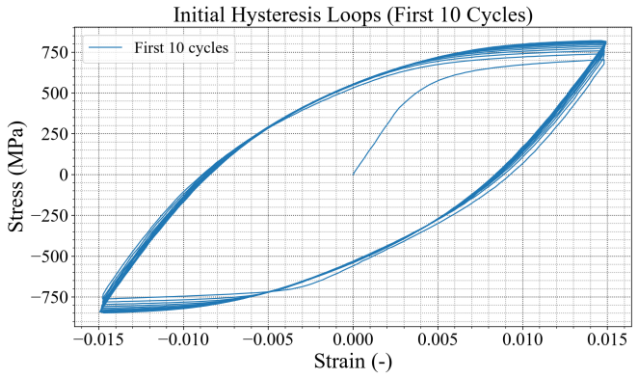


Figure 4 First 10 hysteresis loops

and the user is prompted to define a stress range in which the behavior is linear, and the Young's modulus is estimated using linear regression

```
Determination of Youngs Modulus
Define the region for linear regression (Youngs modulus)!
Enter value for Limit_bottom (MPa) [5]: 5
Enter value for Limit_upper (MPa) [250]: 250
```

```
Strain has been shifted by (%): -0.0011
Youngs modulus E (MPa): 151334.2
Is it OK (Y/N) [Y]:
```

with its corresponding calculation, display of the result, and confirmation. The selected region is also plotted as in Figure 5

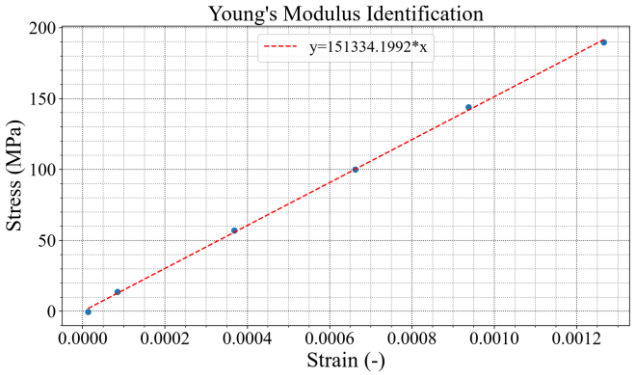


Figure 5 Selected linear section useful for the elastic modulus calculation

The user can then use the calculated Young's modulus, the value automatically read from the configuration file (E_0 from *.json*), or enter a custom value that will be used in subsequent calculations (by selecting M-modify).

```
Would you like to use evaluated value 151334.2 MPa (Y/N/M-modify) - when N, E0=150000 MPa is used [Y]:
```

Then, the plastic work is calculated for each cycle (based on the defined elastic modulus)

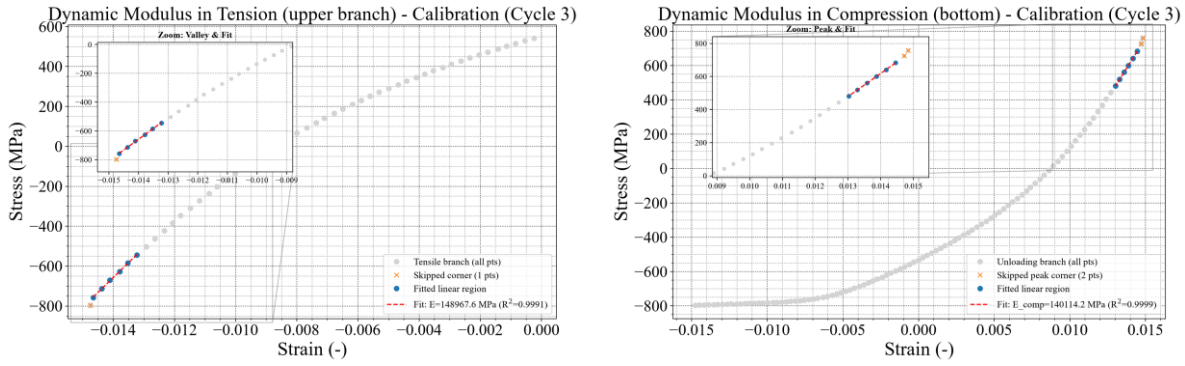
```
Calculating plastic work (vectorized)...
```

Next, the calculation of the dynamic modulus for both the tension (upper) and compression (bottom) branches of each available hysteresis loop is performed

```
Is the linear fit in tension (upper) branch on cycle 3 OK? (Y/N) [Y]: N

Adjust calibration parameters after viewing the plot:
Enter elastic region size [200.0 MPa]:
Enter initial points to skip at reversal corner [2]: 1
Is the linear fit in tension (upper) branch on cycle 3 OK? (Y/N) [Y]:
```

with a graphical display to make it easier to choose the linear segment, as shown in Figure 6.



a) b)

Figure 6 Dynamic modulus for a) the tension (upper) branch and b) the compression (bottom) branch

Then, the user is asked to provide the tensile offset plastic strain value, and 0.05% is typically chosen (the lower the number of samples per cycle the higher the offset strain should be, otherwise the scatter increases):

```
Enter the EQUIVALENT tensile offset plastic strain [0.0001]: 0.0005
```

It can happen that the trend of the cyclic yield stress will be scattered, for some reasons, so an option to select a validity stress range is also implemented allowing outliers to be removed

```
Would you like to specify a validity range for cyclic yield stress? (Y/N) [N]:
```

Also, to ensure that the half-life hysteresis loop will not be distorted, an option to select the previous cycle is implemented

```
Target cycle 210 might be distorted. Use previous cycle 200 instead? (y/N) [n]:
```

The number of cycles to crack initiation (N_f^{E10}) is evaluated based on a predefined relative deviation (typically $\pm 10\%$) of the dynamic compression modulus (from the bottom branch of the hysteresis loop) from the baseline linear regression trend established during the quasi-stabilized regime around half-life $[N_1, N_2]$ ($E_{trend}(N) = k_{trend} \cdot N + q_{trend}$):

```
Cycles to failure (criterion reached within measured data): Nf_10 = 400
```

After all calculations have been completed the computational time necessary for the entire procedure is displayed in the command window

```
================================================================
BENCHMARK: Active CPU computation time: 15.861 s
================================================================
```

Then, the user has the option to generate a pdf protocol

```
Do you want to generate the post-processing record protocol (PDF)? (Y/N) [Y]:
```

And finally, there is an option to save the custom .json configuration file for further analysis

```
Do you want to save the settings from this run as a new JSON configuration template? (Y/N) [N]:
```

Outputs

All files generated by the *LCF_single_eval* workflow are saved inside a dedicated output folder:

$$/Output_[MaterialName]_[ControlMode]/$$

The exported CSV files are described below:

output_half.csv	Contains primary stabilized mid-life ($N_f/2$) material parameters (refer to the nomenclature table in the introduction for detailed symbol definitions), including stress and strain amplitudes, plastic strain, fatigue lives, dynamic moduli, cyclic yield stress, and dissipated plastic work.

Evaluation_input.csv	Records single-specimen configuration settings and test limits: user-selected N_1 and N_2 regression bounds, reference Young's modulus (E), plastic work reference modulus, linear elastic region limit ($\Delta\sigma_{LIN}$), and offset plastic strain.
amp_mean_values.csv	Full cycle-by-cycle evolution history tracking how stress amplitude, strain amplitude, mean stress, and dynamic moduli change progressively from cycle 1 to failure (N_f).
Upper_peaks.csv	Cycle-by-cycle record tracking only the upper turning points (peaks) of the mechanical response. Used to monitor cyclic hardening, softening, and mean stress relaxation over the test duration.
half_life_history.csv	Detailed diagnostic log focused strictly on the stabilized mid-life region around $N_f/2$ (or its immediate neighbor), ensuring true values used for multi-specimen models are taken from a stable plateau. True stress and true strain tensor components.
For_FEM.csv	Formatted and reduced cyclic data mapped specifically for direct import into Finite Element Analysis (FEA) solvers or structural fatigue software for component-level fatigue life prediction.
entire_history.csv	Complete time-series log of the test containing engineering stress and full uniaxial tensor components, formatted for structural FEM export and continuous history tracking (from $N = 1$ to $N = N_f$).

and for the figures:

Dynamic modulus-Tension.png	Evolution of the tensile (from upper branch of hysteresis loop) dynamic modulus over the specimen's lifetime.
Dynamic modulus-Compression.png	Evolution of the compressive (from bottom branch of hysteresis loop) dynamic modulus over the specimen's lifetime.
Dynamic modulus - Tension - fit calibration cycle.png	Linear regression fit used to calibrate and extract the dynamic modulus on the tensile branch.
Dynamic modulus - Compression - fit calibration cycle.png	Linear regression fit used to calibrate and extract the dynamic modulus on the compressive branch.
Dynamic modulus - comparison.png	Side-by-side comparison tracking both tension and compression dynamic moduli degradation.
Cyclic yield stress.png	Tracking of the cyclic yield stress (Y_{cyc}) evolution across test cycles.
Cyclic ye vs Amp Mean stress.png	Relationship between cyclic yield stress and stress amplitude/mean stress variations.
Amplitude-mean stress.png	Evolution and interaction of stress amplitudes against mean stresses.
Amplitude-mean strain.png	Evolution and interaction of strain amplitudes against mean strains.

Amplitude_axial_plastic_strain.png	History of the plastic strain amplitude component throughout the fatigue test.
Young's Modulus.png	Identification of Young's modulus from the initial loading history.
WP_vs_N.png	Dissipated plastic strain energy per cycle (ΔW_p) plotted against the number of cycles (N).
stress_strain_time_history.png	Time-series signals showing recorded stress and strain histories (first 10 cycles).
SigMaxSigMin_vs_N.png	Maximum and minimum peak stresses (sigma_max, sigma_min) tracked over all cycles.
Nf.png	Failure criterion determination based on the 25% stress amplitude drop (N_f).
initial_hysteresis_loops_10_cycles.png	Stress-strain response across the first 10 cycles showing initial hardening/softening.
HystLoopHalflife_Comparison.png	Comparative overlay of two hysteresis loops at the mid-life.
HystLoopHalflife.png	Stabilized stress-strain hysteresis loop extracted at mid-life ($N_f/2$).
Hysteresis_Loop.png	Representative single-cycle or overall stress-strain hysteresis loop.
Header_logo.png	Institutional graphic asset (logo) rendered at the top of emulator PDF protocols.
E_compression_Nf10_determination.png	Failure criterion determination based on a 10% change in compression modulus (N_f^{E10}).

Stress controlled

To characterize fatigue behavior, where ratcheting can affect fatigue life, under load controlled operating conditions, laboratory tests are executed under stress control. This Python tool provides an end-to-end evaluation of raw machine data ingestion, cycle by cycle amplitude extraction, ratcheting strain accumulation analysis, and a real-time plot pipeline visualization. It also handles tracking of strain amplitude and ratcheting progression, as well as automated failure detection concluding with standardized data export for downstream fatigue life evaluation.

Files information

First of all, before starting to use the script, it is necessary to check if every file is located in the working directory (project folder), here is a list of them:

- LCF_single_eval.py — main script
- Protocol_Ratcheting.py — support script
- Config_Ratchet_In718SLMZ.json — configuration template
- AllData750MPa100MPa.csv — experimental raw data
- Header_logo.png — support figure
- S105008959299.bib (optional) — additional reference

The structure of the input file is discussed in the previous chapter.

User interface

After running the script in the Python interpreter (Spyder is recommended; scripts have been tested on Python 3.14.6), the first prompt asks the user to select an existing *.json* configuration file from the accompanying files.

```
================================================================================
     UNIFIED LCF & RATCHETING TOOL (FABER CA 23109)
================================================================================
Select existing JSON configuration profile via dialog? (Y/N) [Y]:
```

Throughout the script execution, default suggestions derived from the configuration file are displayed in square brackets, allowing the user to simply press Enter to accept them. Once the configuration is loaded, the option to select the test method becomes available.

```
Configuration mode: 'Stress-controlled'
Select test mode:
  1 = Strain-controlled (LCF)
  2 = Stress-controlled (Ratcheting)
Choice [2]: 2
```

Subsequently, the material name and its reference elastic modulus (E_0) are retrieved, and a graphical file dialog opens to let the user select the raw experimental data file from the universal testing machine

```
Active material: 'In718' | Reference E0: 150000 MPa

--------------------------------------------------------------------------------
SELECT RAW TEST DATA FILE (.CSV or .TXT)
```

with the possibility of choosing a bibliographic citation:

```
--> Found bibliography file: 'S0142112326001799.bib'
  Use 'S0142112326001799.bib' for protocol citation? (y/n) [y]:
--------------------------------------------------------------------------------
```

To facilitate a quick verification, the first three rows are displayed to inspect and confirm the delimiter and decimal separator of the document:

```
========================================================================
FILE: AllData750MPa100MPa.csv
------------------------------------------------------------------------
HEADER:      Time (s);Total Strain (%);Plastic Strain (%);Stress (MPa);Load (kN);Position (mm);Cycle Number
DATA ROW 1:  0;0.00217;0.002851;0.054208;0.001538;-1.191844;1
DATA ROW 2:  0.0752;0.008612;0.184023;13.956172;0.395918;-1.185747;1
========================================================================
```

This is followed by a step to confirm the detected columns and confirm the name of the output folder, with additional information then being requested, such as the unit format.

```
DETECTED COLUMNS IN FILE:
  [0] Time (s)
  [1] Total Strain (%)
  [2] Plastic Strain (%)
  [3] Stress (MPa)
  [4] Load (kN)
  [5] Position (mm)
  [6] Cycle Number

CONFIRM COLUMN ASSIGNMENTS (Press [Enter] to keep default, or enter index/name):
Confirm column for Time [Time (s)]:
Confirm column for Force / Stress [Stress (MPa)]:
Confirm column for Deformation / Strain [Total Strain (%)]:
Confirm column for Cycle Number [Cycle Number]:
```

then confirming the name of the output folder

```
Enter output folder name [Output_In718_Stress_Control_Ratcheting]:
```

Additional information is requested, such as the unit format:

```
Are data in the form of force/elongation (Y/N) - if N, stress and strain are expected directly [N]:
Is strain in percent (%)? (Y/N) [Y]:
```

Then, the number of cycles to failure is calculated and shown in the command window, according to a 25% decrease in stress amplitude.

```
Number of cycles to failure: 164
```

To ensure the data is consistent, the first few rows of the data are checked,

```
========================================================================
INSPECTION OF INITIAL LOADING DATA (ZEROING / EXTENSOMETER TARE REMOVAL)
========================================================================
Total skipped ROWS so far: 1 rows from raw CSV (rows_to_skip in JSON)).
Please, interrupt the evaluation and change the value in JSON if initial data is missing!
First stress peak is at row index: 55 (852.6 MPa).
[INFO] No sharp tare jump detected. Loading appears smooth from Row 0.
Enter number of initial rows to trim. Write 0 to finish [0]:
```

and the first branch of the hysteresis loop is displayed, shown in Figure 7.

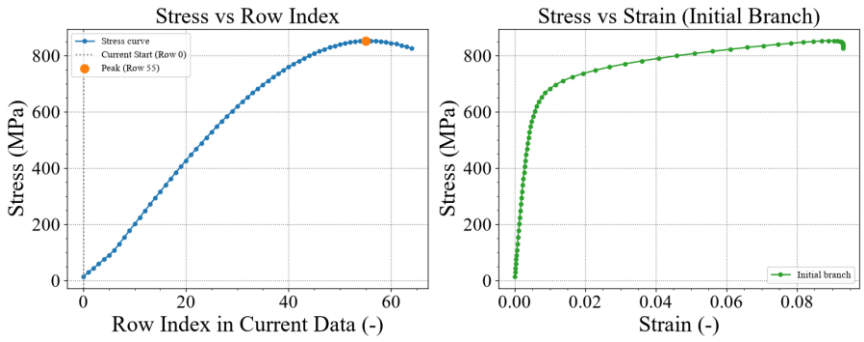


Figure 7 Initial data displayed for cutting

A stress range is requested in which the behavior is linear, and the elastic Young's modulus is estimated using linear regression

```
Determination of Youngs Modulus
Define the region for linear regression (Youngs modulus)!
Enter value for Limit_bottom (MPa) [5]:
Enter value for Limit_upper (MPa) [250]:
.........................................
Strain has been shifted by (%): -0.0024
Youngs modulus E (MPa): 150015.9
Is it OK (Y/N) [Y]:
```

with its corresponding calculation, display of the result, and confirmation. The selected region is also plotted as in Figure 8.

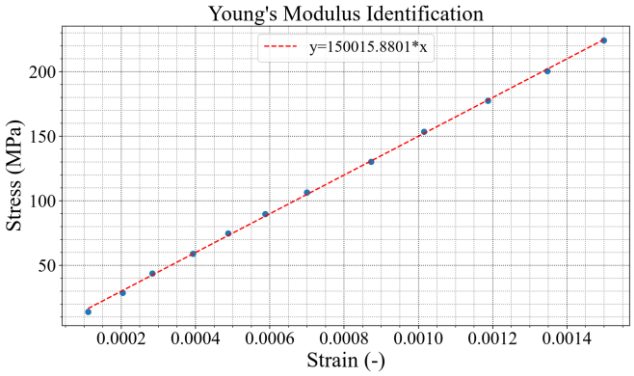


Figure 8 Selected linear section useful for the elastic modulus calculation

User can then use the calculated Young's modulus, the value automatically read from the configuration file, or enter a custom value that will be used in subsequent calculations

```
Would you like to use evaluated value 150015.9 MPa (Y/N/M-modify) - when N, E0=150000 MPa is used [Y]: N
```

then, the plastic work is calculated for each cycle

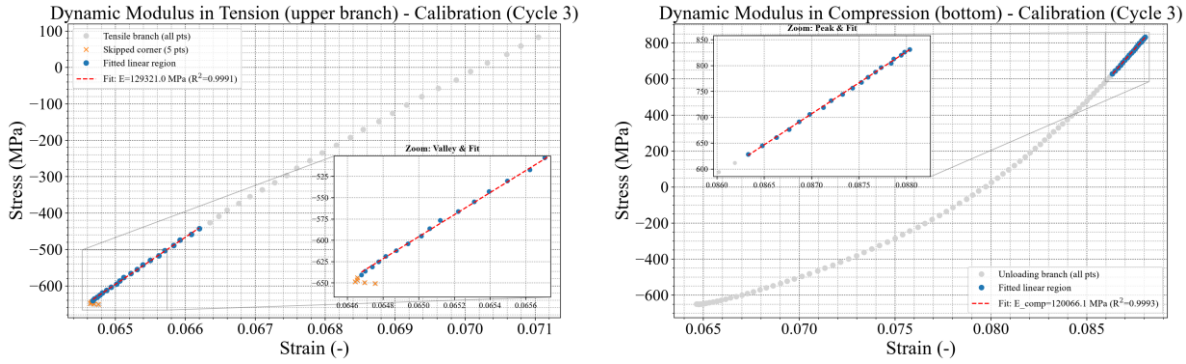
```
Calculating plastic work (vectorized)...
```

Next, the calculation of the dynamic modulus for both the tension and compression branches is performed

```
Is the linear fit in tension (upper) branch on cycle 3 OK? (Y/N) [Y]: N

Adjust calibration parameters after viewing the plot:
Enter elastic region size [200.0 MPa]:
Enter initial points to skip at reversal corner [2]: 5
Is the linear fit in tension (upper) branch on cycle 3 OK? (Y/N) [Y]:
```

with a graphical display to make it easier to choose the linear segment, as shown in Figure 9.



a) b)

Figure 9 Dynamic modulus for a) the tension (upper) branch and b) the compression (bottom) branch

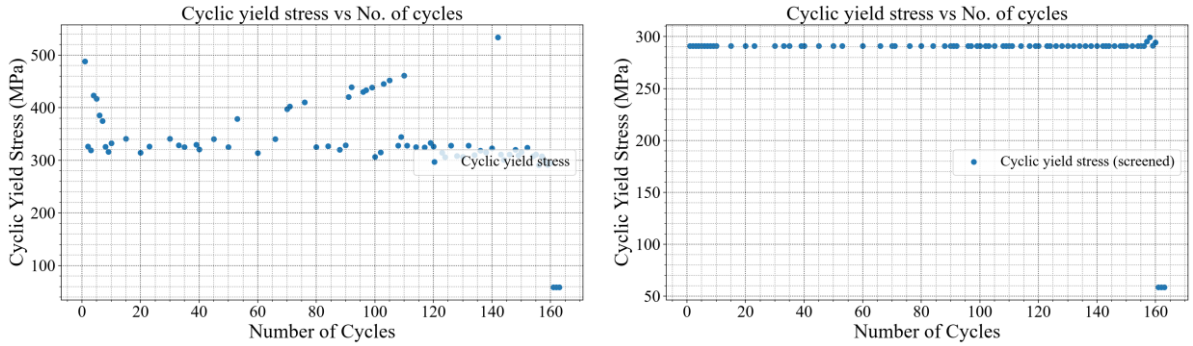
Then, the user is prompted to provide the tensile offset plastic strain value, and 0.05% is typically chosen

```
Enter the EQUIVALENT tensile offset plastic strain [0.0001]: 0.0005
```

After calculating the cyclic yield stress, a validity range can be applied to ensure a smooth trend

```
Would you like to specify a validity range for cyclic yield stress? (Y/N) [Y]: Y
Current cyclic yield stress values range from 58.5 MPa to 533.8 MPa
Enter minimum valid cyclic yield stress (MPa) [40.0]:
Enter maximum valid cyclic yield stress (MPa) [300.0]:
Is this validity range OK? (Y/N) [Y]:
```

The Figure 10 shows how the trend changed after the cut.



a) b)

Figure 10 Cyclic yield stress trend a) before specifying a validity range and b) after the correction

After that, a plot appears showing the mean strain curve, which can be used to identify the linear region, shown in Figure 11.

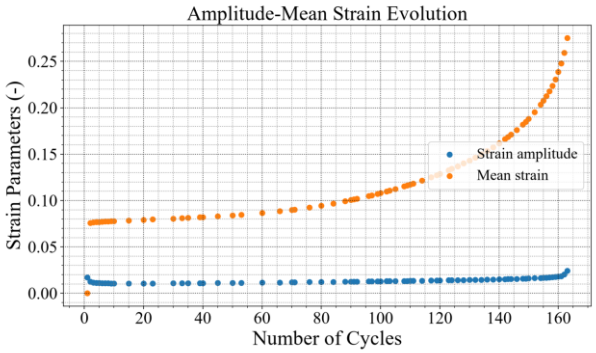


Figure 11 Strain amplitude and mean strain evolution

This is followed by the possibility of selecting a range where the linear regression method will be used:

```
PLEASE, DEFINE THE RANGE FOR RATCHETING RATE EVALUATION!
Enter value for N1_RAT [20]:
Enter value for N2_RAT [40]:
Is it OK (Y/N) [Y]:
```

and the selected region is also plotted as in Figure 12.

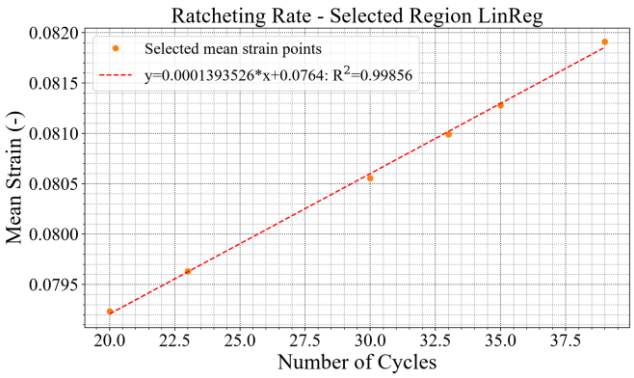


Figure 12 Selected linear region useful for the ratcheting rate calculation

To define the range for half-life value evaluation, a specific interval is selected where linear regression is applied

```
PLEASE, DEFINE THE RANGE FOR HALF-LIFE VALUES EVALUATION (INCLUDING HALF LIFE)!
Enter value for N1 [70]:
Enter value for N2 [90]:
Cycles to failure (criterion reached within measured data): Nf_10 = 126
```

After all calculations have been completed, the computational time necessary for entire procedure is displayed in the command window

```
================================================================
BENCHMARK: Active CPU computation time: 15.636 s
================================================================
```

Then, there is a possibility to generate a pdf protocol

```
Do you want to generate the post-processing record protocol (PDF)? (Y/N) [Y]:
```

and finally, to save the custom *.json* configuration file for further analysis

```
Do you want to save the settings from this run as a new JSON configuration template? (Y/N) [N]:
```

Outputs

All files generated by the *LCF_single_eval* workflow are saved inside a dedicated output folder:

$$/Output_[MaterialName]_[ControlMode]/$$

The exported CSV files are described below:

output_half.csv	Contains primary stabilized mid-life ($N_f/2$) material parameters (refer to the nomenclature table in the introduction for detailed symbol definitions), including stress and strain amplitudes, plastic strain, fatigue lives, dynamic moduli, cyclic yield stress, dissipated plastic work, and ratchet strain rate.
Evaluation_input.csv	Records single-specimen configuration settings and test limits: user-selected N_1^{RAT}, N_2^{RAT} and N_1,N_2 regression bounds respectively, reference Young's modulus (E), plastic work reference modulus, linear elastic region limit ($\Delta\sigma_{LIN}$), and offset plastic strain during the load-controlled run.

amp_mean_values.csv	Full cycle-by-cycle evolution history tracking how stress amplitudes, strain amplitudes, mean strains, and ratcheting progression change progressively from cycle 1 to failure (N_f).
half_life_history.csv	Detailed diagnostic log focused strictly on the stabilized mid-life region around $N_f/2$ (or its immediate neighbor), verifying stable load-displacement behavior.
For_FEM.csv	Formatted and reduced cyclic data mapped specifically for direct import into Finite Element Analysis (FEA) solvers or structural fatigue software for component-level fatigue life prediction.
entire_history.csv	Complete time-series log of the test containing engineering stress and full uniaxial tensor components, formatted for structural FEM export and continuous history tracking (from $N = 1$ to $N = N_f$).

and for the figures:

Dynamic modulus - comparison.png	Side-by-side comparison tracking both tension and compression dynamic moduli degradation under load control.
Cyclic yield stress.png	Tracking of the cyclic yield stress (Y_{cyc}) evolution across test cycles.
Cyclic ye vs Amp Mean stress.png	Relationship between cyclic yield stress and stress amplitude/mean stress variations.
Amplitude-mean stress.png	Checking applied stress amplitudes against mean stresses during testing.
Amplitude-mean strain.png	Evolution and interaction of resulting strain amplitudes against mean strains (reflecting ratcheting).
Young's Modulus.png	Identification of Young's modulus from the initial loading history.
WP_vs_N.png	Dissipated plastic strain energy per cycle (ΔW_p) plotted against the number of cycles (N).
stress_strain_time_history.png	Time-series signals showing recorded stress and strain histories (first 10 cycles).
SigMaxSigMin_vs_N.png	Maximum and minimum peak stresses (sigma_max, sigma_min) tracked over all cycles to monitor load stability.
Ratcheting_rate_regression.png	Linear regression analysis tracking the steady cyclic creep (ratcheting) strain rate per cycle ($d\varepsilon_m/dN$).
initial_hysteresis_loops_10_cycles.png	Stress-strain response across the first 10 cycles showing initial cyclic stabilization or ratcheting.
HystLoopsChosen.png	Selected hysteresis loops plotted at specific intervals throughout the fatigue life (hysteresis loops can be specified in *json* file - "Representative_Loops_Fractions").
HystLoopHalflife.png	Representative stress-strain hysteresis loop extracted at mid-life ($N_f/2$).
Hysteresis_Loop.png	Overview of overall stress-strain hysteresis loops.

Header_logo.png	Institutional graphic asset (logo) rendered at the top of emulator PDF protocols.
E_compression_Nf10_determination.png	Failure criterion determination based on modulus or compliance changes under stress control.
Dynamic modulus-Tension.png	Evolution of the tensile (from upper branch of hysteresis loop) dynamic modulus over the specimen's lifetime.
Dynamic modulus-Compression.png	Evolution of the compressive (from bottom branch of hysteresis loop) dynamic modulus over the specimen's lifetime.
Dynamic modulus - Tension - fit calibration cycle.png	Linear regression fit used to calibrate and extract the dynamic modulus on the tensile branch.
Dynamic modulus - Compression - fit calibration cycle.png	Linear regression fit used to calibrate and extract the dynamic modulus on the compressive branch.

e-N curve evaluation – conventional vs 3D method (*lcf_3d_eval*)

The classic or conventional approach applies three independent linear regressions in bi-logarithmic coordinates:

- Basquin equation (elastic strain amplitude vs. life)
- Manson–Coffin equation (plastic strain amplitude vs. life)
- Ramberg–Osgood equation (cyclic stress–strain curve)

Because these models are fitted independently, the resulting cyclic stress-strain parameters (K', n') do not necessarily satisfy the compatibility conditions with the lifetime parameters:

$$K' \neq \frac{\sigma_f'}{\varepsilon_f'^{\,n'}} \qquad\qquad n' \neq \frac{b}{c}$$

To overcome this limitation and ensure mathematical compatibility between cyclic plasticity and fatigue life parameters, a unified 3D method was proposed [1, 2]. The core principle is applying a single 3D linear regression in the coordinate space $\log_{10}(\varepsilon_{\mathrm{ap}}) - \log_{10}(\sigma_{\mathrm{a}}) - \log_{10}(2N_{\mathrm{f}})$. A dedicated Python tool has been developed to implement this 3D identification workflow, coupled with automated post-processing and generation of a standardized PDF protocol.

Files information

First of all, before starting using the script, it is necessary to check if every file is located in the working directory (project folder), here is a list of them:

- lcf_3d_eval.py	main script
- Protocol.py	support script
- Inputs.txt	input values
- Header_logo.png	support figure
- S0142112326001799.bib (optional)	additional reference

The input file *Inputs.txt* from which the values will be read has the following structure:

884.95, 866.86, …	σ_a	[MPa]
0.01248, 0.00997, …	ε_a	[MPa]
358, 387, …	N_f	[cycles]
1.00, 1.00, …	w	[-]
210000	E_{const}	[MPa]
217904.91, 217713.77, …	E_{var}	[MPa]
20.91762, 14.12159, …	W_p	[MPa]
-17.22, -18.01, …	σ_m	[MPa]
Nf_E10	% decrease info	[-]

These values are written manually from the output *output_half.csv* file obtained from the previous analysis.

Alternatively, to avoid manual data transcription, the automated preprocessing script *2_eN/Input_preparation.py* can be utilized. This tool automatically gathers individual specimen results (from *output_half.csv* files across multiple test folders), filters them according to the selected fatigue life criterion (N_f or N_f^{E10}), and compiles the complete *Inputs.txt* file ready for the 3D evaluation script.

Workflow using the preparation script:

1. Ensure that individual test directories contain their respective *output_half.csv* files generated by *LCF_single_eval.py*.
2. Run *Input_preparation.py*.
3. Select the preferred fatigue life criterion (N_f or N_f^{E10}) when prompted.
4. The script will automatically generate the formatted *Inputs.txt* file in the working directory.

User interface

After running the main script, a file selection dialog will appear to choose the *Inputs.txt* file. Following this, information regarding the material and the case number will be requested, which is used to generate the name of the output folder

```
--> Selected input file: D:/                                                    .txt

Enter Material name [          ]:
Enter Case number [2]: 1
--> All outputs will be saved to: Output_          _Case_1
```

as well as the information for the protocol

```
--------------------------------------------------------------------------------
 METADATA & POST-PROCESSING RECORD (Press Enter to keep default)
--------------------------------------------------------------------------------
  Experimenter [                                ]:
  Post-processing [                                ]:
  Experiment type / Loading [                                                ]:
  Sample count / specifications [8]:
  Date of experiment [2025]:
  Date of post-processing [25.3.2026]:
  Notes [1%/s, Extensometer - gauge length 10 mm. Criterion: Nf_E10]:
--------------------------------------------------------------------------------
```

with the possibility of choosing a bibliographic citation

```
--> Found bibliography file: '                    '
  Use '                .bib' for citation? (y/n) [y]:
```

In the case the citation file cannot be found, it can be entered manually

```
--------------------------------------------------------------------------------
 No .bib file selected for footer citation.
 Do you want to enter reference details manually? (y/n) [n]: Y

 Enter citation specifications (press Enter to leave an optional field empty):
   1. Authors (e.g.           et al.):
   2. Year (e.g. 2025):
   3. Article Title:
   4. Journal / Book title:
   5. Volume / Issue (optional):
   6. Pages / Article ID (optional):
   7. DOI or URL (optional):
--------------------------------------------------------------------------------
```

After that, the calculation will begin, followed by the automatic saving of the output files.

Outputs

All files generated by the e-N post-processing workflow are saved inside a dedicated folder

$$/Output_[MaterialName]_Case_[CaseNumber]/$$

The exported CSV files are described below:

mechprop_3D_const.csv	Low-cycle fatigue and cyclic plasticity parameters identified via the 3D method using the elastic constant modulus E_{const}
mechprop_3D_var.csv	Low-cycle fatigue and cyclic plasticity parameters identified via the 3D method using different dynamic modulus for each strain level ($E_{\text{var}} = (E_{upp} + E_{btm})/2$)
mechprop_conv_const.csv	Constants determined using the Conventional method with E_{const}
mechprop_conv_var.csv	Constants determined using the Conventional method with E_{var}
Reliability.csv	Coefficients of determination (R^2) comparing all models.
Parameters.txt	Readable text summary of primary 3D parameters (E_{const}) along with both 3D and conventional methods transition lives
3D_linear_regression.txt	Raw geometric parameters of the 3D linear spatial fit
Fatigue_Params_4P.csv	Parameters of the 4-parametric fatigue model for both SWT and plastic strain energy W_p
Page2_Input_Table.csv	Specimen-level energy and mean stress data (used to populate Page 2 of the protocol)
tests_single_column.csv	Internal metadata vector containing experimenter, dates, specimen specs, note string, $N_{t,3\text{D}}$, N_f criterion (nf_type), and $N_{t,2\text{D}}$
citation.txt	Created when citation metadata is entered manually via the command window (contains citation string and clickable DOI URL).

and for the figures:

xy_plane.png	Cyclic Stress–Strain Curve (CSSC): σ_{a}vs. ε_{ap} evaluated strictly with E_{const}
xz_plane.png	Coffin–Manson–Basquin (CMB) Curve: Elastic, plastic, and total strain curves plotted with E_{const}
3D.png / 3D_comparison_Econst_vs_Evar.png	3D Spatial Regression: Experimental data points and directional line in the $\log_{10}(\varepsilon_{\text{ap}}) - \log_{10}(\sigma_{\text{a}}) - \log_{10}(2N_{\text{f}})$ space. Second one displays E_{const}vs. E_{var}

page2_cssc_var.png	CSSC Curve (E_{var}): σ_{a}vs. $\varepsilon_{\mathrm{ap}}$evaluated using specimen-specific dynamic modulus E_{var} (hollow markers and dashed fits).
page2_cm_comparison.png	Complete CMB Comparison (E_{const}vs. E_{var}): All strain components ($\varepsilon_{\mathrm{ae}}$, $\varepsilon_{\mathrm{ap}}$, ε_{a}) vs. N_f. Direct visual comparison of solid lines/filled markers (E_{const}) against dashed lines/hollow markers (E_{var}).
page2_energy_plots.png	Energy Models vs. N_f: with 4-parametric fits.

Examples and Verification Cases

To demonstrate the capabilities of the FABER LCF and ratcheting evaluation toolchain, the package includes pre-configured example datasets derived from recent experimental campaigns published in literature [3, 4]. These cases cover both strain-controlled LCF, stress-controlled ratcheting, and multi-specimen e-N curve evaluations.

Uniaxial Ratcheting and LCF Cases (Inconel 718)

Two benchmark datasets originating from the experimental work by Adamec et al. [3] are provided to validate the single-specimen evaluation routines (*LCF_single_eval.py*).

Stress-Controlled Ratcheting Case:

Demonstrates the evaluation of ratcheting strain accumulation, mean stress/strain evolution, and dynamic modulus degradation under asymmetric cyclic loading for additively manufactured Inconel 718 and generate the standardized two-page PDF (*Protocol_Ratcheting.pdf*) evaluation report.

Strain-Controlled LCF Case:

Illustrates the standard low-cycle fatigue evaluation pipeline, including initial Young's modulus identification, plastic work calculation, and stabilized half-life property extraction for additively manufactured Inconel 718 and generate the standardized two-page PDF (*Protocol_LCF.pdf*) evaluation report.

Multi-Specimen e-N Evaluation Case (AlSi10Mg)

A complete multi-specimen dataset taken from Halama et al. [4] is included to showcase the 3D parameter identification workflow (*lcf_3d_eval.py*). This example demonstrates how multiple specimens tested at various strain/stress amplitudes are processed using the automated input preparation script to fit unified 3D fatigue curves (comparing conventional regressions against the advanced 3D method) and generate the standardized two-page PDF (*Protocol_eN.pdf*) evaluation report.

Bibliography

[1] A. Niesłony, C. El Dsoki, H. Kaufmann, and P. Krug, "New method for evaluation of the Manson–Coffin–Basquin and Ramberg–Osgood equations with respect to compatibility," *International Journal of Fatigue*, vol. 30, no. 10–11, pp. 1967–1977, 2008, doi: 10.1016/j.ijfatigue.2008.01.013.

[2] D. Ličková, R. Halama, and Z. Poruba, "Identification of Fatigue Constants by means of 3D Method," *Strojnícky časopis – Journal of Mechanical Engineering*, vol. 66, no. 2, pp. 107–116, 2016, doi: 10.1515/scjme-2016-0025.

[3] T. Adamec, T. Hassan, J. Zapletal, S. K. Kondepati, and F. Šebek, "Backstress shift modelling concept for improving uniaxial ratcheting predictions for wrought 304 stainless steel and additively manufactured Inconel 718," *Eur. J. Mech. A Solids*, vol. 114, p. 105770, 2025, doi: 10.1016/j.euromechsol.2025.105770.

[4] R. Halama, W. Macek, B. Tomiczek, Z. Poruba, F. Sari, J. Rahimi, J. Mesicek, and F. Berto, "Prediction of ratcheting and LCF damage using a memory surface in cyclically hardening additively manufactured AlSi10Mg with enhanced uniaxial ratcheting resistance," *Int. J. Fatigue*, vol. 210, p. 109658, 2026, doi: 10.1016/j.ijfatigue.2026.109658.